Assessments Compilation

Assessment 1 · Assessment 2 · Assessment 3

Topics: Tesla Car Functions · Linear Regression
Supervised & Unsupervised Learning · OpenRouter

ASSESSMENT 1

Tesla Car Functions

Tesla vehicles are built around a deeply integrated software-hardware architecture that enables features far beyond conventional automobiles. Below is a structured overview of Tesla's key functions and systems.

1. Autopilot & Full Self-Driving (FSD)

Tesla's driver-assistance suite uses eight cameras, twelve ultrasonic sensors, and forward-facing radar (older models) combined with the Tesla FSD Computer to deliver semi-autonomous and, progressively, fully autonomous driving.

Feature	Description
Traffic-Aware Cruise Control	Maintains set speed and safe following distance automatically.
Autosteer	Keeps the car centred in a detected lane on highways.
Auto Lane Change	Changes lanes on the driver's signal after a safety check.
Autopark	Parallel and perpendicular parking with minimal driver input.
Summon / Smart Summon	Moves the car to/from a parking spot via the Tesla app.
FSD (Supervised)	Navigate on Autopilot, traffic-light/stop-sign control, city streets.

2. Over-the-Air (OTA) Software Updates

Tesla delivers feature upgrades, bug fixes, and performance improvements wirelessly, similar to a smartphone. Updates can add new Autopilot capabilities, improve range estimates, or even increase motor power without any dealership visit.

3. Energy & Battery Management

- Regenerative Braking – Converts kinetic energy back to battery charge; one-pedal driving possible.
- Battery Pre-conditioning – Warms or cools the pack before DC fast charging for optimal charge speed.
- Trip Planner – Routes automatically through Supercharger stops with real-time state-of-charge estimates.
- Energy App – Displays consumption vs. projected use and historical efficiency data.
- Scheduled Charging / Departure – Charges off-peak and pre-conditions cabin to a set departure time.

4. Infotainment & Display

A large central touchscreen (15.4" on Model 3/Y, 17" on Model S/X) controls nearly every vehicle function. Key capabilities include:

- Navigation powered by Google Maps with live traffic and Supercharger availability.
- Streaming services: Netflix, YouTube, Spotify, Apple Music (while parked or via Wi-Fi/LTE).
- Arcade Games – Tesla Arcade with titles like Cuphead and The Witcher 3 using game controllers.
- Rear-seat entertainment screen (Model S/X Plaid) with HDMI input.
- Voice commands via the integrated microphone for navigation, calls, climate, and more.

5. Safety Systems

System	Function
Collision Avoidance	Automatic Emergency Braking (AEB) + Forward Collision Warning.
Side Collision Warning	Alerts and steers away from vehicles in blind spots.
Pedestrian Detection	Camera-based detection with automatic braking.
Sentry Mode	360° camera surveillance when parked; clips saved to USB.
Dog / Child Mode	Maintains cabin temperature and displays a dashboard notice.
Cabin Camera (Driver Mon.)	Monitors driver attentiveness during Autopilot use.

6. Connectivity & Smart Features

- Tesla App – Remote lock/unlock, climate pre-conditioning, charging status, live camera view.
- Phone Key & Key Card – Passive Bluetooth entry and NFC card backup.
- Wi-Fi & LTE – Always-on connectivity for maps, streaming, and OTA updates.
- Vehicle-to-Home (V2H) & Powerwall Integration – Export stored energy back to the home grid.
- Fleet Telemetry – Anonymised driving data feeds Tesla's neural-network training pipeline.

7. Performance Modes

Tesla offers software-selectable driving modes. Chill Mode limits acceleration for comfort or new drivers; Standard provides everyday performance; Track Mode (Model S Plaid) configures cooling, torque vectoring, and stability control for circuit use. Drag Strip Mode maximises launch control for quarter-mile runs.

ASSESSMENT 2

Linear Regression

Linear regression is one of the most fundamental supervised learning algorithms. It models the relationship between a dependent variable (target) and one or more independent variables (features) by fitting a straight line (or hyperplane) to the observed data.

1. Core Concept

The goal is to find the line $y = mx + c$ (simple) or the hyperplane $y = w_0 + w_1x_1 + \ldots + w_nx_n$ (multiple) that minimises prediction error across all training examples.

- y – Dependent variable (what we predict).
- x – Independent variable(s) (input features).
- m / w – Slope / weights (how much y changes per unit of x).
- c / w■ – Intercept / bias (y value when all x = 0).
- ε – Error term (residual) capturing noise not explained by the model.

2. Types of Linear Regression

Type	Description
Simple Linear Regression	One independent variable. Example: predicting house price from size.
Multiple Linear Regression	Two or more independent variables. Example: price from size, rooms, location.
Polynomial Regression	Features raised to powers; still linear in parameters.
Ridge Regression (L2)	Adds L2 penalty to prevent overfitting on high-dimensional data.
Lasso Regression (L1)	Adds L1 penalty; drives less-useful coefficients to zero (feature selection).
ElasticNet	Combines L1 + L2 penalties for a balanced regularisation approach.

3. Cost Function – Mean Squared Error (MSE)

Linear regression minimises the Mean Squared Error (MSE) between predicted and actual values:

$$MSE = (1/n) \times \Sigma (y_i - ■_i)^2$$

where n is the number of data points, y_i is the actual value, and $■_i$ is the predicted value.

4. How the Model is Trained

Two main approaches are used to find the optimal weights:

- **Gradient Descent** – Iteratively updates weights in the direction that reduces MSE. The learning rate controls step size. Variants include Batch, Stochastic (SGD), and Mini-Batch Gradient Descent.
- **Normal Equation (Ordinary Least Squares)** – Closed-form solution $w = (X^T X)^{-1} X^T y$. Fast for small datasets; impractical for very large feature spaces.

5. Assumptions of Linear Regression

- Linearity – Relationship between X and y is linear.
- Independence – Observations are independent of each other.
- Homoscedasticity – Constant variance of residuals across all levels of X.
- Normality of Errors – Residuals are approximately normally distributed.
- No Multicollinearity – Independent variables are not highly correlated with each other.

6. Evaluation Metrics

Metric	Meaning
MSE	Mean Squared Error – average of squared residuals. Penalises large errors.
RMSE	Root MSE – same unit as y; easier to interpret.
MAE	Mean Absolute Error – average of absolute residuals; robust to outliers.
R^2	Coefficient of Determination – proportion of variance explained (0–1; higher = better).
Adjusted R^2	R^2 penalised for the number of predictors; useful for model comparison.

7. Real-World Applications

- House Price Prediction – area, bedrooms, location → price.
- Sales Forecasting – advertising spend → revenue.
- Medical Dosage – body weight → drug dosage.
- Stock Price Modelling – economic indicators → stock return.
- Energy Consumption – temperature, occupancy → kWh usage.

8. Python Implementation (Scikit-learn)

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(r2_score(y_test, y_pred))
```

ASSESSMENT 3

Supervised & Unsupervised Learning + OpenRouter

Part 1 – Supervised vs Unsupervised Learning

Machine learning algorithms are broadly categorised by the type of signal available during training. The two most fundamental paradigms are supervised and unsupervised learning.

A. Supervised Learning

In supervised learning the model is trained on **labelled data** — each input example is paired with the correct output. The model learns a mapping from inputs to outputs and is evaluated by how accurately it predicts labels on unseen data.

Property	Details
Labelled Data Required	Yes – every training sample has an input–output pair.
Goal	Learn a function f(x) ≈ y.
Feedback	Direct: loss is computed against known labels.
Output type	Discrete (classification) or continuous (regression).

Common Supervised Learning Algorithms

- Linear / Logistic Regression
- Decision Trees & Random Forests
- Support Vector Machines (SVM)
- k-Nearest Neighbours (k-NN)
- Gradient Boosting (XGBoost, LightGBM)
- Neural Networks / Deep Learning

Where Supervised Learning is Used

Use Case	How It Works
Email Spam Detection	Classify emails as spam / not spam using labelled training examples.
Image Recognition	Label images (cat, dog, car) – used in self-driving, medical imaging.
Sentiment Analysis	Determine if a review is positive, negative, or neutral.

Medical Diagnosis	Predict disease presence from patient records and test results.
Credit Scoring	Predict loan default risk from applicant financial history.
Speech Recognition	Transcribe audio to text using labelled audio-text pairs.
Fraud Detection	Flag suspicious transactions based on historical fraud labels.
Price Prediction	Estimate house / stock prices from historical labelled datasets.

B. Unsupervised Learning

In unsupervised learning the model is given **unlabelled data** and must discover hidden structure, patterns, or groupings on its own. There are no predefined correct answers.

Property	Details
Labelled Data Required	No – model explores raw data without guidance.
Goal	Find structure, clusters, associations, or compressed representations.
Feedback	Indirect: intrinsic metrics (inertia, silhouette score).
Output type	Clusters, embeddings, anomaly scores, association rules.

Common Unsupervised Learning Algorithms

- K-Means Clustering
- Hierarchical Clustering (Agglomerative / Divisive)
- DBSCAN – Density-Based Spatial Clustering
- Principal Component Analysis (PCA) – Dimensionality Reduction
- Autoencoders – Neural-network-based compression & anomaly detection
- Apriori / FP-Growth – Association Rule Mining
- Gaussian Mixture Models (GMM)
- t-SNE / UMAP – Visualisation of high-dimensional data

Where Unsupervised Learning is Used

Use Case	How It Works
Customer Segmentation	Group customers by behaviour for targeted marketing (K-Means).
Recommendation Systems	Discover latent user/item factors for collaborative filtering.

Anomaly / Fraud Detection	Identify unusual patterns without needing labelled fraud examples.
Document Clustering	Group news articles or research papers by topic.
Gene Expression Analysis	Cluster genes with similar expression patterns in bioinformatics.
Image Compression	PCA / Autoencoders reduce image dimensions while preserving features.
Market Basket Analysis	Apriori finds items frequently bought together (retail).
Network Intrusion Detection	Detect novel cyber-attacks by flagging statistical anomalies.

Side-by-Side Comparison

Aspect	Supervised | Unsupervised
Training Data	Labelled (input + output pairs) | Unlabelled (input only)
Objective	Predict output for new inputs | Discover hidden structure
Evaluation	Accuracy, F1, RMSE | Silhouette score, inertia
Examples	SVM, Neural Nets, Random Forest | K-Means, PCA, DBSCAN
Applications	Spam filter, OCR, price forecast | Customer clustering, compression

Part 2 – OpenRouter: Usage of Models

OpenRouter is a unified API gateway that provides a single endpoint to access hundreds of large language models (LLMs) from multiple providers — including OpenAI, Anthropic, Google, Meta, Mistral, and many open-source models — through one consistent interface.

What is OpenRouter?

- A middleware layer between developers and AI model providers.
- Single API key grants access to 200+ models from dozens of providers.
- OpenAI-compatible API format – existing code needs minimal changes.
- Automatic fallback routing if a model or provider is unavailable.
- Pay-per-token pricing aggregated across all providers.
- Free tier available for many open-source models (e.g. Llama 3, Mistral 7B).

How to Use OpenRouter

```
Base URL: https://openrouter.ai/api/v1
```

```
Header: Authorization: Bearer YOUR_OPENROUTER_KEY
Header: HTTP-Referer: https://your-site.com (required)
```

```
import openai
client = openai.OpenAI(
base_url='https://openrouter.ai/api/v1',
api_key='YOUR_OPENROUTER_KEY'
)
response = client.chat.completions.create(
model='anthropic/claude-3.5-sonnet',
messages=[{'role': 'user', 'content': 'Hello!'}]
)
```

Popular Models Available on OpenRouter

Model ID	Description
openai/gpt-4o	OpenAI's multimodal flagship model.
anthropic/claude-sonnet-4-5	Anthropic's latest efficient model.
google/gemini-2.0-flash	Google's fast, cost-effective multimodal model.
meta-llama/llama-3.3-70b	Meta's open-source 70B parameter model.
mistralai/mistral-7b-instruct	Lightweight open-source model; often free tier.
deepseek/deepseek-r1	Strong reasoning model from DeepSeek.
microsoft/phi-3-medium	Microsoft's compact but capable model.
cohere/command-r-plus	Optimised for RAG and enterprise workflows.

Key Features of OpenRouter

- **Model Routing** – Automatically pick the cheapest or fastest model meeting your criteria.
- **Fallback Chains** – If primary model is down, traffic shifts to a backup automatically.
- **Streaming** – Server-sent events (SSE) streaming supported across all models.
- **Function Calling / Tool Use** – Supported for compatible models (GPT-4o, Claude, Gemini).
- **Prompt Caching** – Reduce costs by caching repeated system-prompt tokens.
- **Usage Analytics** – Dashboard showing token consumption, cost, and latency per model.
- **Rate-Limit Handling** – Distributes load across provider replicas to minimise throttling.

OpenRouter vs Direct Provider APIs

Aspect	OpenRouter | Direct API
API Keys	One key for all models | Separate key per provider
Model Access	200+ models via one endpoint | Only that provider's models

Pricing	Slight markup on some models | Direct provider pricing
Reliability	Built-in fallback routing | Single point of failure
Use Case	Prototyping, multi-model apps| Production single-model systems

Practical Use Cases

- Rapid Prototyping – Switch between GPT-4o and Claude without rewriting API calls.
- Cost Optimisation – Route simple queries to cheap models; complex ones to powerful models.
- A/B Testing Models – Run the same prompt through multiple models and compare outputs.
- High-Availability Chatbots – Fallback chains ensure near-zero downtime.
- Research & Benchmarking – Easily test and compare dozens of models in one script.

End of Assessments